A Project report on

# JSON REFINER ADVANCED EDITION

Under Manonmaniam Sundaranar University

Submitted by


| | |
|---|---|
| **JENIHA J** | **(23083300500112012)** |
| **JENISHA ANTO J** | **(23083300500112013)** |
| **JESNI HAMIO G** | **(23083300500112014)** |
| **KEERTHANA P** | **(23083300500112015)** |
| **LAKSHMI V** | **(23083300500112016)** |

**DEPARTMENT OF COMPUTER APPLICATIONS**

**ARUNACHALA ARTS & SCIENCE (WOMEN) COLLEGE**

Manonmaniam Sundaranar University

Tirunelveli-627 001

# JSON Refiner Advanced Edition

Professional JSON Processing Tool with Gradio Interface

## Project Description

JSON Refiner Advanced Edition is an intelligent platform designed to simplify the process of working with JSON data. It is developed using Python's powerful data processing capabilities and provides features such as JSON parsing, validation, transformation, and refinement. The system uses an interactive and visually appealing interface built with Gradio, making it easy for users to process and manage JSON data efficiently. The platform helps solve common issues related to JSON formatting and data quality by offering AI-based JSON generation, automatic data type detection, schema validation, case conversion, flattening and unflattening, and clean production-ready output.

The system works by analyzing unstructured key-value data and automatically identifying the correct data types, such as strings, numbers, booleans, null values, arrays, and objects. It also ensures that the JSON structure follows consistent naming styles like snake_case, camelCase, kebab-case, and PascalCase. In addition, the platform can validate JSON data using schemas and provide different transformation options to improve the structure and usability of the data.

Through its user-friendly Gradio interface, JSON Refiner Advanced Edition makes JSON processing simple, organized, and efficient. It helps users handle complex JSON transformations while maintaining data accuracy, proper validation, and high-quality output suitable for real-world applications.

## Scenarios

### Scenario 1: Converting CSV Data to JSON

A data analyst wants to convert a CSV file that contains mixed data types into a well-structured JSON format. The system takes key-value pairs from the CSV file, such as name: John, age: 28, active: true, score: 95.5, and automatically identifies the correct data types. It then organizes the data using consistent naming styles and produces a clean JSON output that is ready for real-world use. The analyst can also validate the generated JSON with a predefined schema to make sure the data is accurate before transferring it to a database.

### Scenario 2: Standardizing API Responses

A backend developer works with JSON responses from different third-party APIs, but each API uses different key naming styles and structures. The system helps by converting all keys into a single consistent format (for example converting FirstName, first_name, and firstName into one naming style). It can also combine multiple API responses into one structured format, remove empty or null values, and check the data using a master schema. This ensures the final data is clean and ready to be used by the frontend without extra manual processing.

### Scenario 3: Managing Configuration Files

A DevOps engineer often works with configuration files written in simple text format and needs to convert them into JSON for automated deployment systems. The system reads configuration values such as database_host*: localhost, port: 5432,* ssl_enabled*:* true, retry_count*: 3* and automatically identifies the correct data types. It checks for required fields, prevents duplicate keys, and provides clear error messages if something is wrong. Finally, it generates valid JSON that can be easily used in infrastructure automation tools while keeping configurations easy to maintain.

### Scenario 4: Flattening Data for Analysis

An analytics team sometimes receives JSON files with deeply nested structures that are difficult to analyze in spreadsheet tools. The system simplifies this by flattening the nested data. For example, a structure like user: {name: "John", profile: {age: 28}} can be converted into a flat format such as user.name: "John" and user.profile.age: *28*. This makes the data easier to use in tools like CSV or Excel. When needed, the data can also be converted back to its original nested format without losing any information.

**Scenario 5: Validating Data Using Schemas**

A quality assurance team needs to make sure that JSON data follows company standards. The system allows them to validate JSON data using a predefined JSON Schema. It checks whether the structure and required fields are correct and provides clear error messages if there are missing or incorrect values. This process helps prevent invalid data from entering the system and ensures that all data remains consistent across different applications.

## Architecture Overview

JSON Refiner Advanced is built as a modular platform combining Python backend with sophisticated JSON processing algorithms for intelligent data transformation. The architecture prioritizes data integrity, user experience, and production readiness through Gradio's interactive interface and comprehensive validation mechanisms.

## Core Technologies

- Python 3.8+: Core programming language for robust data processing
- Gradio 4.0+: Interactive web interface framework for user interaction
- JSON5: Advanced JSON parsing for flexible input handling
- JSONSchema: Schema validation and data conformance checking
- Regular Expressions: Pattern matching for advanced parsing
- Google Colab: Cloud-based development and deployment environment
- HTML5/CSS3: Modern web standards for interface rendering

## Pre-requisites

### Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- pip: Python package manager (comes with Python)
- Web Browser: Chrome, Firefox, Safari, or Edge

- Internet Connection: Required for Colab and Gradio sharing

**Knowledge Prerequisites**

- Python Basics: Functions, dictionaries, list comprehensions, string manipulation
- JSON Fundamentals: JSON structure, types, and validation
- Regular Expressions: Pattern matching for data parsing
- Gradio Framework: Understanding of interface components and callbacks
- Data Validation: Familiarity with JSON Schema and validation concepts
- Google Colab: Basic notebook interface and cell execution

**Hardware Requirements**

- Processor: Any modern processor (Intel i3 or equivalent)
- RAM: Minimum 2GB local (Colab provides cloud resources)
- Storage: 500MB free space for application and generated files
- Display: Minimum 1024x768 resolution for interface usability

**Project Workflow**

**Phase 1: Environment Setup & Core Architecture**

**Activity 1.1: Install Dependencies & Configure Environment**

- Create requirements.txt with all dependencies
- Set up Google Colab environment with proper settings
- Install required packages: gradio, json5, jsonschema, python-dotenv, pydantic
- Verify all packages are installed correctly

**Activity 1.2: Design Core Data Structures**

- Define CaseStyle Enum for case conversion options
- Create JSONRefinerCore class with all processing methods
- Design type inference system for automatic data type detection
- Implement error tracking and logging mechanisms

**Phase 2: Core JSON Processing Engine**

**Activity 2.1: Implement Type Inference System**

- Create infer_type() method for automatic type detection
- Handle boolean conversions (true/false, yes/no, on/off)
- Implement null value handling (null, none, nil, n/a)
- Parse numbers (integers and floats)
- Process arrays and objects from JSON strings
- Default to string for unrecognized values

**Activity 2.2: Build Key Normalization & Case Conversion**

- Implement to_snake_case() conversion
- Implement to_camelCase() conversion
- Implement to_kebab_case() conversion

- Implement to_pascal_case() conversion
- Create normalize_key() method that applies selected case style
- Test all case conversion combinations

**Phase 3: Validation & Transformation Features**

**Activity 3.1: Implement JSON Schema Validation**

- Create validate_json_schema() method
- Use jsonschema library for Draft7 validation
- Implement error message formatting
- Test against various schema definitions

**Activity 3.2: Build Required Fields Checking**

- Create check_required_fields() method
- Implement field presence validation
- Generate missing field reports
- Combine with schema validation results

**Activity 3.3: Develop JSON Transformation Methods**

- Implement flatten_json() for nested to flat conversion
- Implement unflatten_json() for dot-notation reversal
- Create merge_json_objects() for deep merging
- Implement remove_null_values() for data cleanup
- Test all transformation methods with complex data

**Phase 4: Gradio Interface Development**

**Activity 4.1: Design Core Refining Tab**

- Create input fields for key-value text
- Add case style radio button selector

- Implement remove nulls and flatten checkboxes
- Build JSON output code display
- Add status and statistics output panels

**Phase 5: Integration, Testing & Deployment**

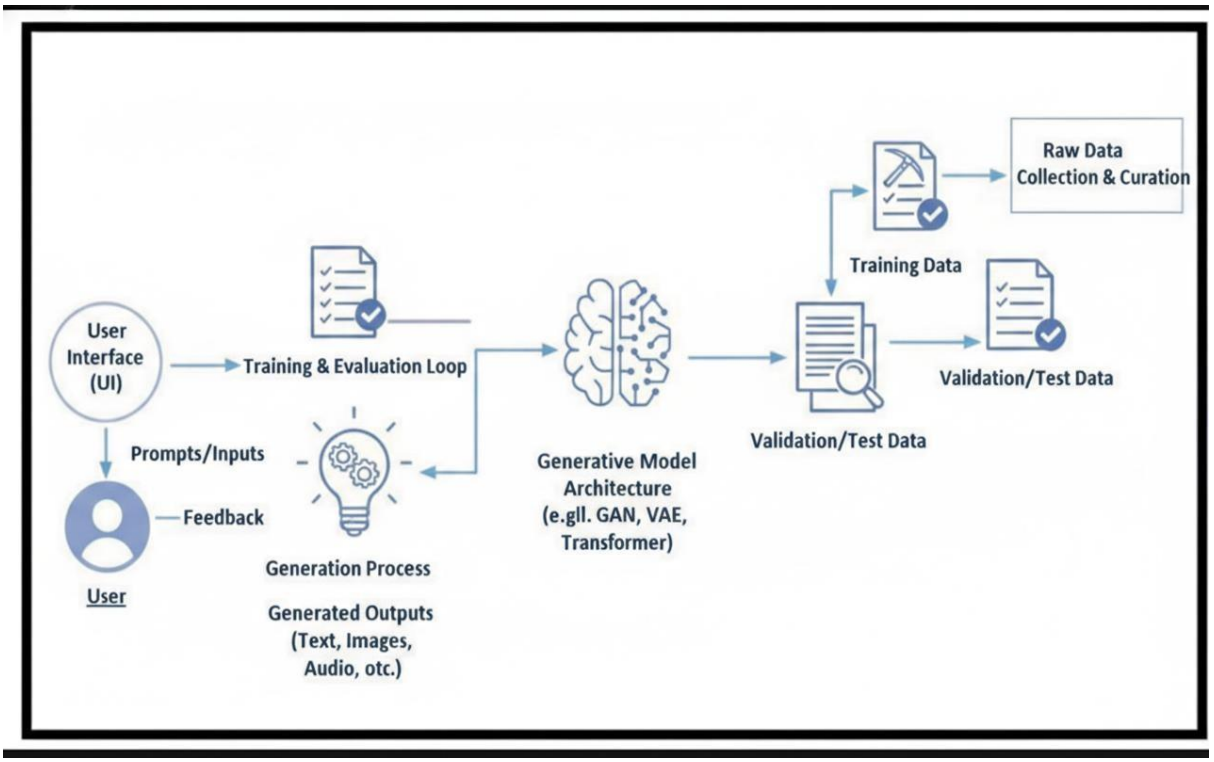
**Activity 5.1: Google Colab Deployment**

- Connect all interface tabs
- Test data flow between components
- Verify transformation pipeline
- Test error handling

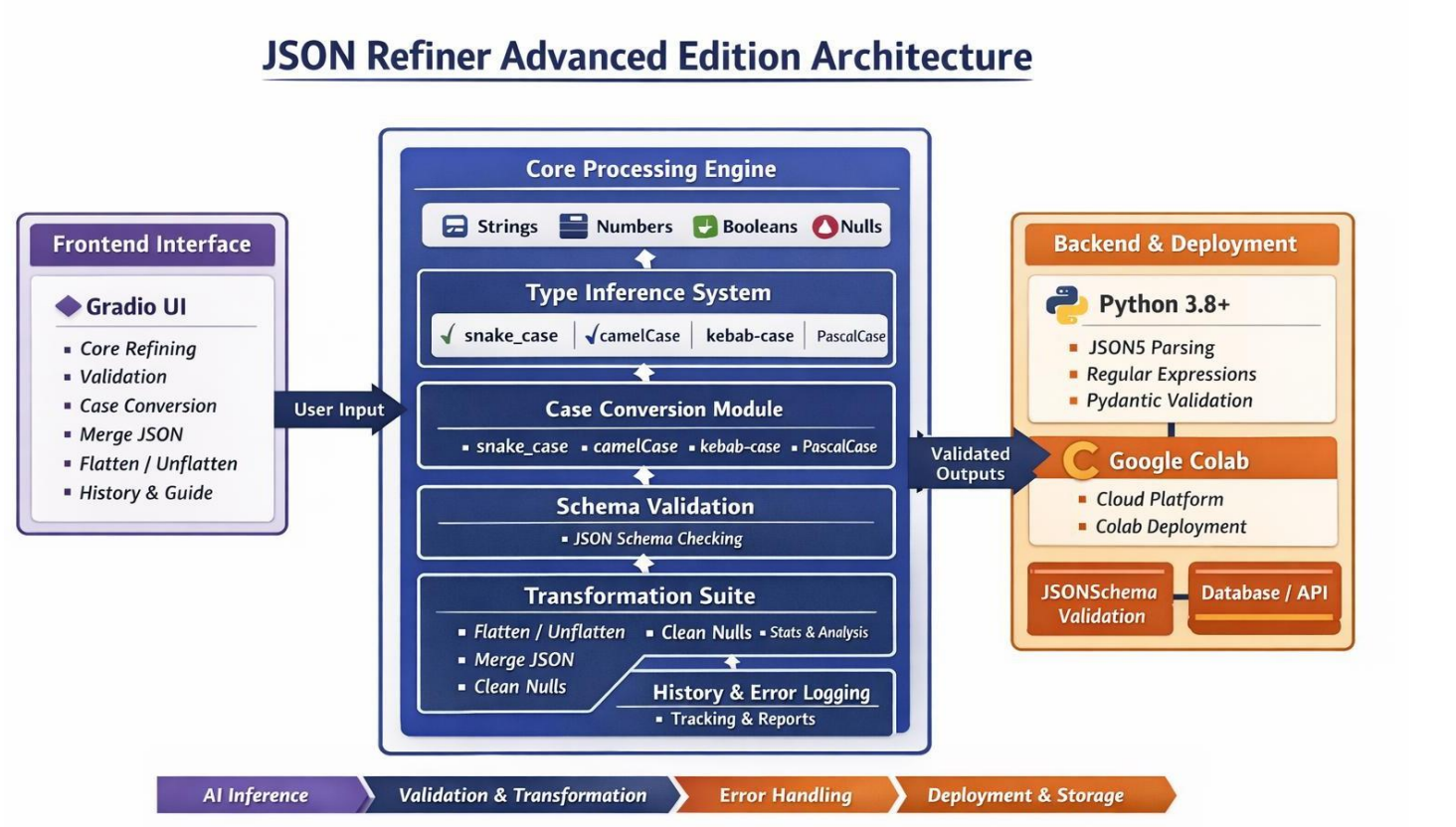
**Activity 5.2: Comprehensive Testing**

- Test all case conversion styles
- Validate type inference accuracy
- Test schema validation with various schemas
- Test merging with conflicting keys
- Test flattening with deeply nested objects

## Technical Architecture




## Project Structure



## MILESTONE 1: Environment Setup & Core Architecture

### Activity 1.1: Install Dependencies

### Load the model :

```
!pip install -U transformers
```

Local Inference on GPU

Model page: https://huggingface.co/ibm-granite/granite-3.3-2b-instruct

⚠ If the generated code snippets do not work, please open an issue on either the model repo and/or on huggingface.js 🙏

```
# Use a pipeline as a high-level helper
from transformers import pipeline

pipe = pipeline("text-generation", model="ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
pipe(messages)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in
```

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
  messages,
  add_generation_prompt=True,
  tokenize=True,
  return_dict=True,
  return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))
```

```
Loading weights: 100% ■■■
```

```
!pip install -q gradio json5 jsonschema python-dotenv pydantic

import gradio as gr
import json
import re
import ast
from datetime import datetime
from pathlib import Path
from typing import Dict, List, Any, Tuple, Optional
import json5
from jsonschema import validate, ValidationError, Draft7Validator
from enum import Enum

# ==============================================================================
# JSON REFINER - ADVANCED VERSION WITH PREMIUM FEATURES (FIXED)
# ==============================================================================
```

**Activity 1.2: Initialize Enum and Core Class**

Create CaseStyle Enum:

```
class CaseStyle(Enum):
    SNAKE_CASE = "snake_case"
    CAMEL_CASE = "camelCase"
    KEBAB_CASE = "kebab-case"
    PASCAL_CASE = "PascalCase"
```

Initialize JSONRefinerCore class with tracking:

```
class JSONRefinerCore:
    def __init__(self):
    self.conversion_log = []
    self.validation_errors = []
    self.transformation_history = []
```

```python
# ==============================================================================
# JSON REFINER - ADVANCED VERSION WITH PREMIUM FEATURES (FIXED)
# ==============================================================================

class CaseStyle(Enum):
    """Case conversion styles"""
    SNAKE_CASE = "snake_case"
    CAMEL_CASE = "camelCase"
    KEBAB_CASE = "kebab-case"
    PASCAL_CASE = "PascalCase"

class JSONRefinerCore:
    """Advanced JSON parsing and refinement engine"""

    def __init__(self):
        self.conversion_log = []
        self.validation_errors = []
        self.transformation_history = []
```

**MILESTONE 2: Type Inference & Case Conversion System**

**Activity 2.1: Implement Type Inference**

The type inference system automatically detects and converts values:

- Booleans: Recognizes true, false, yes, no, on, off, 1, 0
- Nulls: Handles null, none, nil, undefined, n/a
- Numbers: Detects integers and floating-point numbers
- Arrays: Parses JSON arrays from string format
- Objects: Parses JSON objects from string format
- Strings: Default fallback for unrecognized values

Example: "age: 28" → {"age": 28} (integer)
Example: "active: true" → {"active": true} (boolean)

```python
            return JSONRefinerCore.to_pascal_case(key)
        return key

    @staticmethod
    def infer_type(value: str) -> Any:
        """Infer and convert value to correct JSON type"""
        value = value.strip()

        if value.lower() in ['null', 'none', 'nil', 'undefined', 'n/a', 'na']:
            return None

        if value.lower() in ['true', 'yes', 'on', '1']:
            return True
        if value.lower() in ['false', 'no', 'off', '0']:
            return False

        try:
            if '.' not in value:
                return int(value)
        except ValueError:
            pass
```

**Activity 2.2: Build Case Conversion Methods**

Implement four case conversion standards:

1. snake_case: user_name, first_name, is_active
2. camelCase: userName, firstName, isActive
3. kebab-case: user-name, first-name, is-active
4. PascalCase: UserName, FirstName, IsActive

Each method uses regex patterns to normalize input:
- Remove non-alphanumeric characters
- Split on boundaries (spaces, underscores, hyphens)
- Apply case transformation
- Validate output format

```python
def convert_case(json_text: str, target_case: str) -> Tuple[str, str]:
    """Convert JSON keys to different case styles"""
    try:
        data = json.loads(json_text)
        case_enum = CaseStyle[target_case.upper().replace('-', '_')]

        def convert_keys(obj):
            if isinstance(obj, dict):
                return {
                    refiner.normalize_key(k, case_enum): convert_keys(v)
                    for k, v in obj.items()
                }
            elif isinstance(obj, list):
                return [convert_keys(item) for item in obj]
            return obj

        converted = convert_keys(data)
        output = json.dumps(converted, indent=2, ensure_ascii=False)
        return output, f"✅ Converted to {target_case}!"
    except Exception as e:
        return "", f"❌ Error: {str(e)}"
```

## MILESTONE 3: Validation & Transformation Features

### Activity 3.1: JSON Parsing & Validation

Key-value parsing system:

```python
    @staticmethod
    def parse_key_value_text(text: str, case_style: CaseStyle = CaseStyle.SNAKE_CASE) -> Tuple[Dict, List[str]]:
        """Parse key-value text into dictionary"""
        result = {}
        errors = []
        duplicates = []

        lines = text.split('\n')

        for line_num, line in enumerate(lines, 1):
            line = line.strip()

            if not line or line.startswith('#'):
                continue

            if ':' not in line:
                errors.append(f"Line {line_num}: No colon separator found - '{line}'")
                continue

            try:
                key, value = line.split(':', 1)
                key = key.strip()
```

**Activity 3.2: Schema & Field Validation**

Dual validation approach:

1. Required Fields: Check for mandatory fields
   - Input: "name, age, email"
   - Result: Lists missing fields

2. Schema Validation: Validate against JSON Schema
   - Input: {"type": "object", "properties": {...}}
   - Result: Full structural validation
   - Supports Draft 7 standard

```python
    @staticmethod
    def validate_json_schema(data: Dict, schema: Dict) -> Tuple[bool, List[str]]:
        """Validate JSON against schema"""
        errors = []
        try:
            validate(instance=data, schema=schema)
            return True, []
        except ValidationError as e:
            return False, [str(e)]
        except Exception as e:
            return False, [f"Schema validation error: {str(e)}"]

    @staticmethod
    def check_required_fields(data: Dict, required_fields: List[str]) -> Tuple[bool, List[str]]:
        """Check for required fields"""
        errors = []
        missing = []
        for field in required_fields:
            if field not in data:
                missing.append(field)

        if missing:
            return False, [f"Missing required fields: {', '.join(missing)}"]
        return True, []
```

**Activity 3.3: JSON Transformation Operations**

Multiple transformation capabilities:

1. Flatten: Convert nested objects to dot-notation
   Input: {"user": {"name": "John", "age": 28}}
   Output: {"user.name": "John", "user.age": 28}

2. Unflatten: Reverse flattening
   Input: {"user.name": "John", "user.age": 28}
   Output: {"user": {"name": "John", "age": 28}}

3. Merge: Deep merge multiple JSON objects
4. Clean: Remove null/undefined values
5. Stats: Generate detailed statistics

```python
        return "", f"❌ Error: {str(e)}"

def flatten_json_func(json_text: str) -> Tuple[str, str]:
    """Flatten JSON structure"""
    try:
        data = json.loads(json_text)
        flattened = refiner.flatten_json(data)
        output = json.dumps(flattened, indent=2, ensure_ascii=False)
        return output, "✅ JSON flattened!"
    except Exception as e:
        return "", f"❌ Error: {str(e)}"

def unflatten_json_func(json_text: str) -> Tuple[str, str]:
    """Unflatten JSON structure"""
    try:
        data = json.loads(json_text)
        unflattened = refiner.unflatten_json(data)
        output = json.dumps(unflattened, indent=2, ensure_ascii=False)
        return output, "✅ JSON unflattened!"
    except Exception as e:
        return "", f"❌ Error: {str(e)}"
```

**MILESTONE 4: Gradio Interface Development**

**Activity 4.1:** focuses on designing and implementing a comprehensive multi-functional JSON refinement interface that enables users to validate, transform, analyze, and manage JSON data efficiently through an intuitive tab-based UI.

The Gradio interface includes 7 functional tabs:

1. Core Refining: Convert key-value text to JSON
   - Input: Text with key-value pairs
   - Options: Case style, remove nulls, flatten
   - Output: JSON code, status, statistics

```python
# ============= TAB 1: Core Refining =============
with gr.TabItem("🔧 Core Refining"):
    with gr.Row():
        with gr.Column(scale=1):
            gr.Markdown("### 📝 Input Format")
            text_input = gr.Textbox(
                label="Key-Value Text",
                lines=12,
                value="name: John Doe\nage: 28\nis_active: true\nbalance: 1500.50\nemail: john@example.com\ntags: [\"admin\", \"user
            )

            case_style = gr.Radio(
                ["snake_case", "camelCase", "kebab-case", "PascalCase"],
                value="snake_case",
                label="🎨 Key Case Style"
            )

            with gr.Row():
                remove_nulls = gr.Checkbox(label="Remove Nulls", value=False)
                flatten = gr.Checkbox(label="Flatten", value=False)
                pretty_print = gr.Checkbox(label="Pretty Print", value=True)
```

2. Validation: Validate JSON structure
   - Input: JSON, schema, required fields
   - Output: Validation results

```
[2]
17s
# ============= TAB 2: Validation =============
with gr.TabItem("✅ Validation"):
    gr.Markdown("### 🔍 Validate JSON Structure")

    with gr.Row():
        with gr.Column(scale=1):
            json_to_validate = gr.Code(
                language="json",
                label="JSON to Validate",
                lines=10,
                value='{"name": "John", "age": 28}'
            )

            required_fields_input = gr.Textbox(
                label="Required Fields (comma-separated)",
                value="name, age, email"
            )

        with gr.Column(scale=1):
            schema_input = gr.Code(
                language="json",
```

3. Case Conversion: Convert between case styles
   - Input: JSON
   - Output: Converted JSON in target case

```
[2]
17s
# ============= TAB 3: Case Conversion =============
with gr.TabItem("🎨 Case Conversion"):
    gr.Markdown("### 🔄 Convert JSON Key Cases")

    with gr.Row():
        with gr.Column(scale=1):
            json_for_case = gr.Code(
                language="json",
                label="Input JSON",
                lines=10,
                value='{"firstName": "John", "lastName": "Doe"}'
            )

            target_case = gr.Radio(
                ["snake_case", "camelCase", "kebab-case", "PascalCase"],
                value="camelCase",
                label="Target Case Style"
            )

            case_convert_btn = gr.Button("🔄 Convert", size="lg", variant="primary")
```

4. Merge JSON: Combine multiple objects
   - Input: Three JSON objects
   - Output: Merged result

```
# ============= TAB 4: Merging =============
with gr.TabItem("🔗 Merge JSON"):
    gr.Markdown("### 🔀 Merge Multiple JSON Objects")

    with gr.Row():
        with gr.Column(scale=1):
            json1 = gr.Code(
                language="json",
                label="JSON #1",
                lines=8,
                value='{"name": "John"}'
            )
            json2 = gr.Code(
                language="json",
                label="JSON #2",
                lines=8,
                value='{"age": 28}'
            )
            json3 = gr.Code(
                language="json",
                label="JSON #3 (optional)",
                lines=8,
```

5. Flatten/Unflatten: Transform nesting
   - Input: JSON
   - Output: Transformed structure

```
# ============ TAB 5: Flattening ============
with gr.TabItem("📄 Flatten/Unflatten"):
    with gr.Tabs():
        with gr.TabItem("Flatten"):
            gr.Markdown("### 📦 Flatten Nested JSON")

            with gr.Row():
                flatten_input = gr.Code(
                    language="json",
                    label="Nested JSON",
                    lines=10,
                    value='{"user": {"name": "John", "age": 28}}'
                )
                flatten_btn = gr.Button("Flatten", size="lg", variant="primary")

            flatten_out = gr.Code(
                language="json",
                label="Flattened JSON",
                lines=10
            )
            flatten_status = gr.Markdown()
```

6. History: Track transformations
   - Download: Export history report
   - Clear: Reset history

```
[2]
17s
# ============= TAB 6: History =============
with gr.TabItem("📜 History"):
    gr.Markdown("### 📊 Transformation History")

    with gr.Row():
        history_btn = gr.Button("📥 Download History", size="lg", variant="secondary")
        clear_history_btn = gr.Button("🗑️ Clear History", size="lg", variant="stop")

    history_output = gr.Textbox(
        label="History Report",
        lines=15
    )

    history_status = gr.Markdown()

    history_btn.click(
        download_history,
        outputs=[history_output]
    )

    clear_history_btn.click(
        clear_history,
```

7. Guide: User documentation
   - Examples, tips, and troubleshooting

```
[2]
17s
# ============= TAB 7: Guide =============
with gr.TabItem("📚 Guide"):
    gr.Markdown("""
    # 📖 JSON Refiner - Complete Guide

    ## Input Format
    Use simple **Key: Value** format:
    ```
    name: John Doe
    age: 28
    is_active: true
    balance: 1500.50
    ```

    ## Type Inference
    **Automatic type detection:**
    - `true`, `false`, `yes`, `no` → Boolean
    - `null`, `none`, `undefined` → Null
    - `123`, `456` → Integer
    - `3.14`, `2.5` → Float
    - `[1, 2, 3]` → Array (if valid JSON)
    - `{"key": "value"}` → Object (if valid JSON)
```

**UI/UX Features**

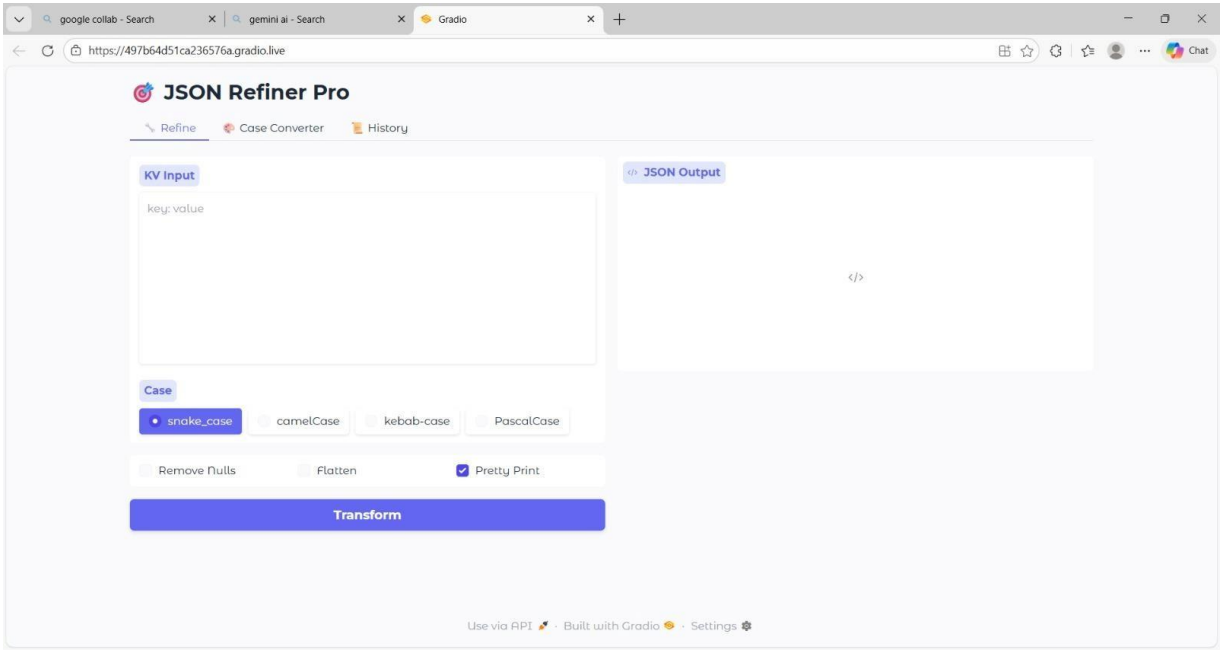
Professional interface design:

- Light theme background
- Color-coded status messages (🟩 success, + error)
- Syntax-highlighted code blocks
- Real-time processing with instant feedback
- Detailed error reporting with line numbers
- Responsive layout for various screen sizes
- Intuitive button-based navigation

**MILESTONE 5: Testing & Deployment**

## Activity 5.1: Google Colab Deployment

```
print("\n✅ JSON Refiner Advanced Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```




## Activity 5.2: Functional Testing

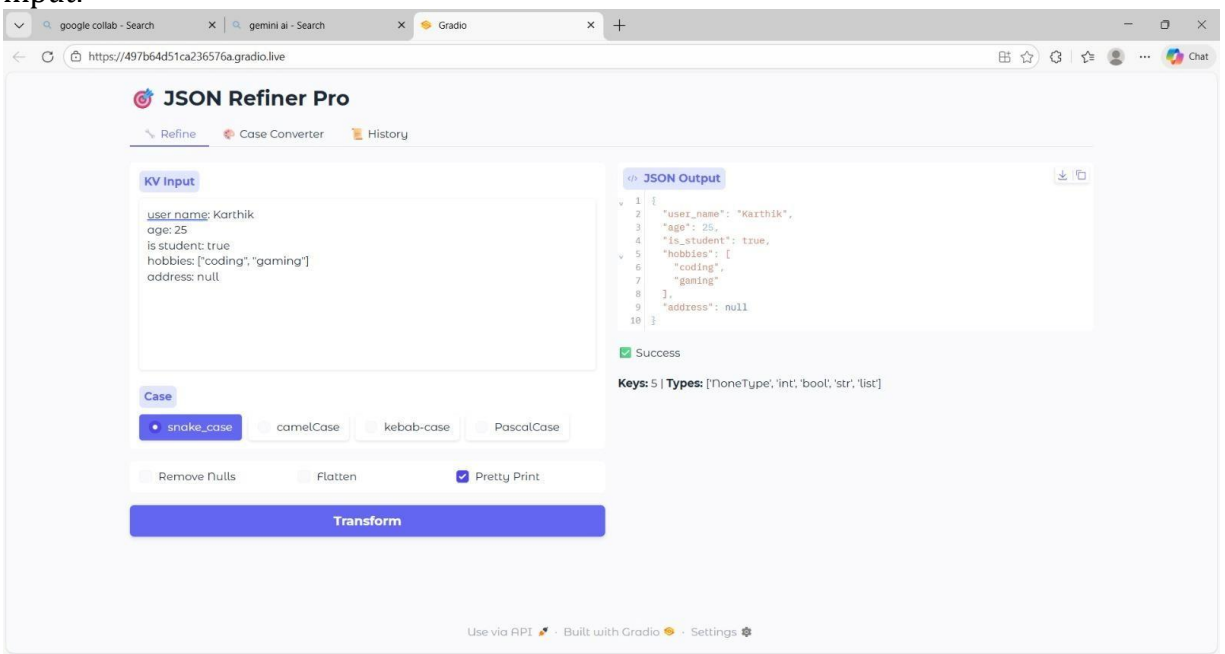
### Test Case 1: Basic Key-Value Conversion

Input:
name: Karthick
age: 25
is student: true

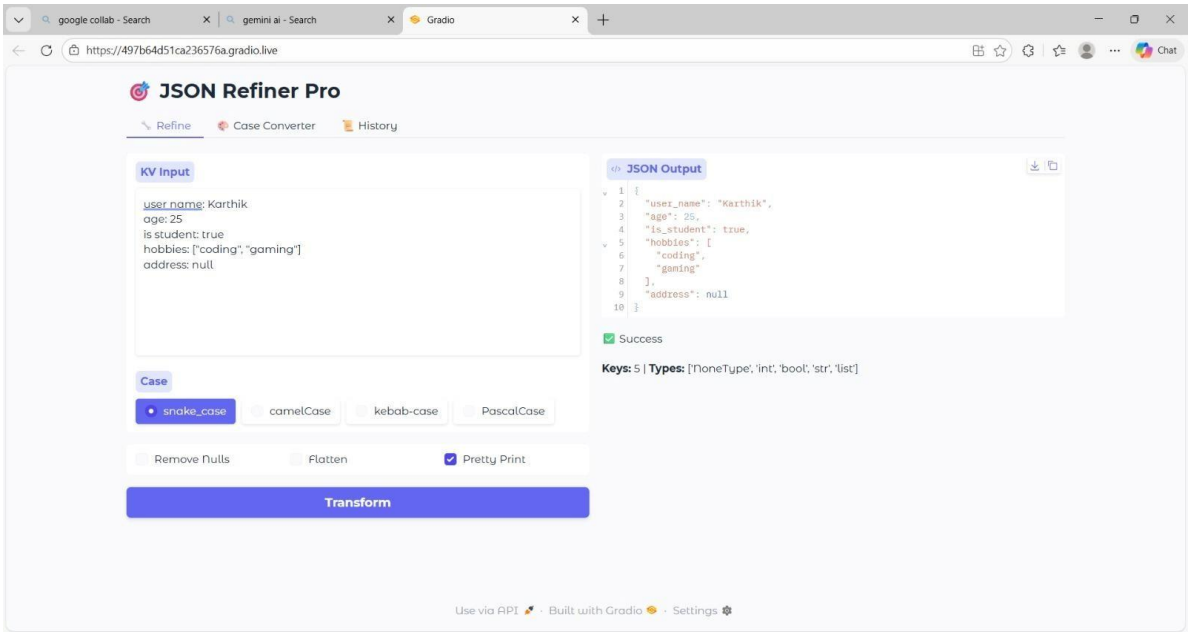
hobbies: coding, gamming

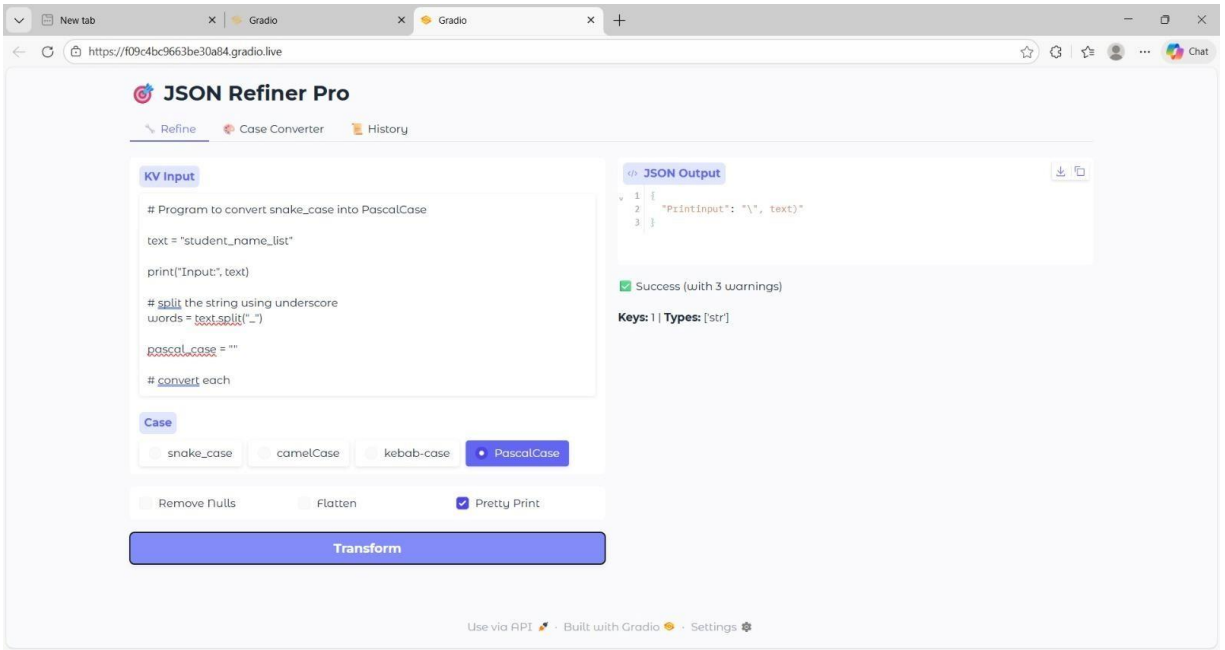
address: null

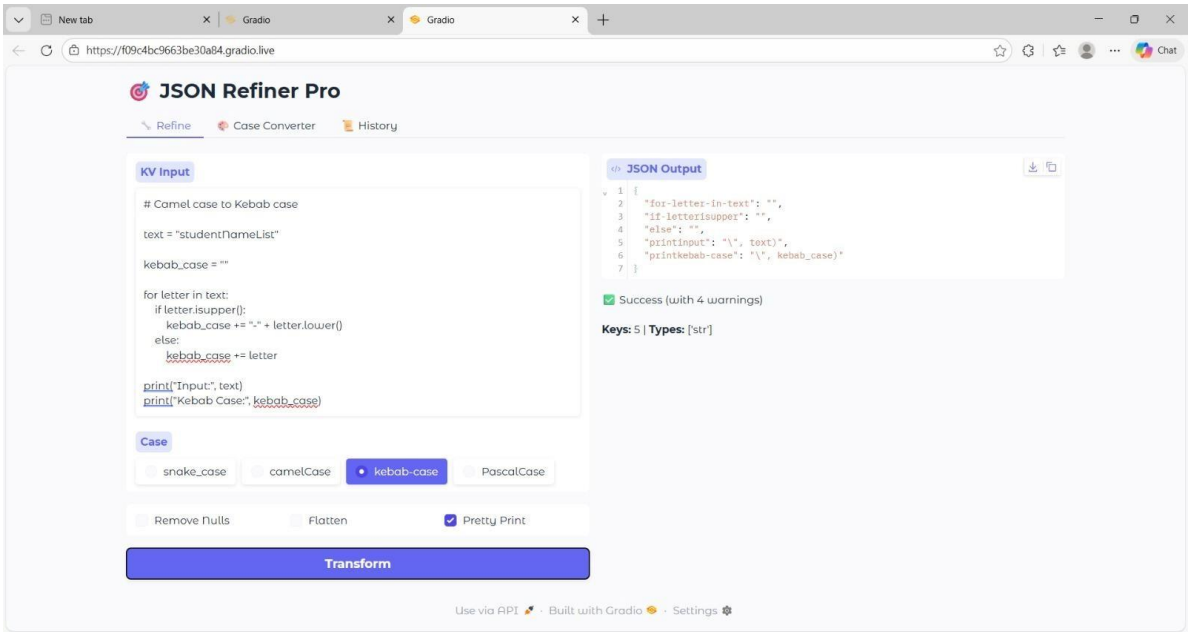
input:

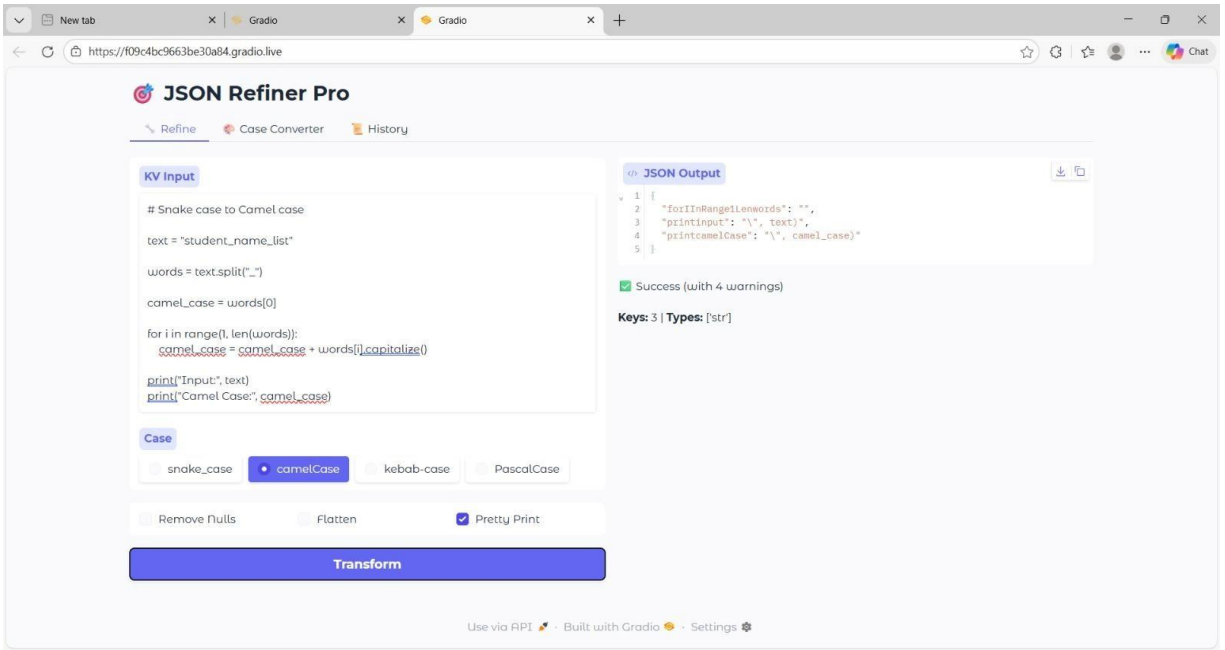


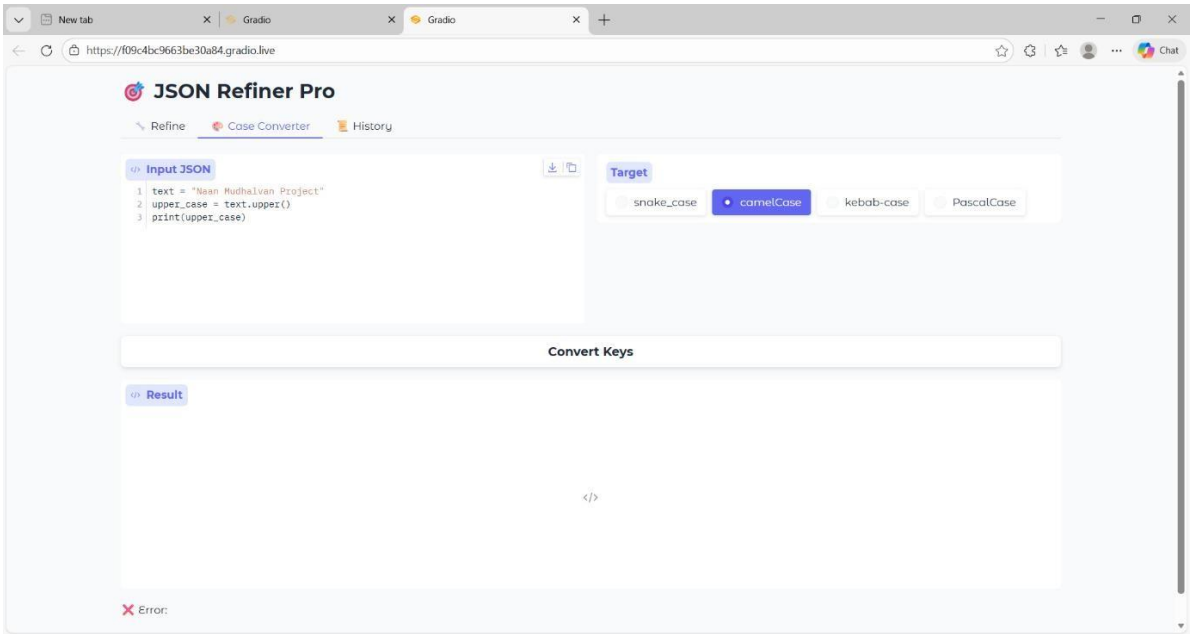

Out put :




New tab
Gradio
Gradio
https://f09c4bc9663be30a84.gradio.live
Chat
JSON Refiner Pro
Refine
Case Converter
History
KV Input
# Program to convert snake_case into PascalCase
text = "student_name_list"
print("Input:", text)
# split the string using underscore
words = text.split("_")
pascal_case = ""
# convert each
Case
snake_case
camelCase
kebab-case
PascalCase
Remove Nulls
Flatten
Pretty Print
Transform
JSON Output
{
"PrintInput": "\", text)"
}
Success (with 3 warnings)
Keys: 1 | Types: ['str']
Use via API · Built with Gradio · Settings


New tab
Gradio
Gradio
https://f09c4bc9663be30a84.gradio.live
Chat
JSON Refiner Pro
Refine
Case Converter
History
KV Input
# Camel case to Kebab case
text = "studentNameList"
kebab_case = ""
for letter in text:
if letter.isupper():
kebab_case += "-" + letter.lower()
else:
kebab_case += letter
print("Input:", text)
print("Kebab Case:", kebab_case)
Case
snake_case
camelCase
kebab-case
PascalCase
Remove Nulls
Flatten
Pretty Print
Transform
JSON Output
{
"for-letter-in-text": "",
"if-letterisupper": "",
"else": "",
"printinput": "\", text)",
"printkebab-case": "\", kebab_case)"
}
Success (with 4 warnings)
Keys: 5 | Types: ['str']
Use via API · Built with Gradio · Settings


New tab
Gradio
Gradio
https://f09c4bc9663be30a84.gradio.live
Chat
JSON Refiner Pro
Refine
Case Converter
History
KV Input
# Snake case to Camel case
text = "student_name_list"
words = text.split("_")
camel_case = words[0]
for i in range(1, len(words)):
camel_case = camel_case + words[i].capitalize()
print("Input:", text)
print("Camel Case:", camel_case)
Case
snake_case
camelCase
kebab-case
PascalCase
Remove Nulls
Flatten
Pretty Print
Transform
JSON Output
{
"forIInRange1Lenwords": "",
"printinput": "\", text)",
"printcamelCase": "\", camel_case)"
}
Success (with 4 warnings)
Keys: 3 | Types: ['str']
Use via API · Built with Gradio · Settings


New tab
Gradio
Gradio
https://f09c4bc9663be30a84.gradio.live
Chat
JSON Refiner Pro
Refine
Case Converter
History
Input JSON
text = "Naan Mudhalvan Project"
upper_case = text.upper()
print(upper_case)
Target
snake_case
camelCase
kebab-case
PascalCase
Convert Keys
Result
Error:

## Conclusion

JSON Refiner Advanced Edition successfully functions as an intelligent JSON processing tool that combines simple data transformation with the strong data handling capabilities of Python. By using Gradio's interactive interface and the cloud environment of Google Colab, the system provides a fast, efficient, and user-friendly way to process JSON data. This makes the platform useful not only for developers but also for data analysts and even users who may not have strong technical knowledge.

The platform shows how AI-assisted data transformation can simplify JSON processing while still maintaining high data quality. It supports features such as automatic data type detection, multiple case conversion styles, advanced transformation operations, detailed validation, and history tracking. These features help users handle complex JSON data easily and produce accurate, clean, and production-ready outputs.

## Future Enhancements

The system is built with a modular design, which makes it easy to expand with additional features in the future. Possible improvements include adding custom transformation rules, regex-based data extraction, integration with popular data analysis tools like Pandas and NumPy, batch processing, API endpoint generation, and the ability to create custom validation rules. Template-based transformations and support for additional naming styles can also be introduced.

In the future, the platform could also include machine learning-based type detection, data anonymization features, and integration with cloud storage services to handle large datasets more effectively.

With further development, JSON Refiner Advanced Edition has the potential to grow into a complete data processing platform that supports multiple data formats while continuing to focus on simplicity, accuracy, and a smooth user experience.

.